1. วัตถุประสงค์ (Purpose)

เอกสารนี้จัดทำขึ้นเพื่อกำหนดมาตรฐานการเขียนโค้ดของระบบ Market Lock Booking System เพื่อให้:

- โค้ดมีความสม่ำเสมอ (Consistency)
- อ่านง่ายและเข้าใจง่าย (Readability)
- บำรุงรักษาได้ง่าย (Maintainability)
- ลดความผิดพลาดจาก Naming ที่ไม่ตรงกัน
- รองรับการพัฒนาแบบทีม

2. Naming Convention Standards

2.1 Class Naming

มาตรฐาน

- ใช้ **PascalCase**
- ใช้ **คำนาม**
- ต้องสอดคล้องกับชื่อ Table

ตัวอย่าง

class AppUser {}

class MarketLock {}

class Booking {}

class Payment {}

เหตุผล

เพื่อให้ Mapping ระหว่าง Code และ Database ชัดเจนและลดความสับสน

2.2 Method Naming

มาตรฐาน

- ใช้ **camelCase**

- ต้องเป็นคำกริยา
- สื่อความหมายชัดเจน

ตัวอย่าง

createBooking(userId, lockId, bookingDate)

approveBooking(bookingId, approvedByUserId)

rejectBooking(bookingId, rejectedReason)

cancelBooking(bookingId)

uploadSlip(bookingId, slipImagePath)

verifyPayment(paymentId)

หลีกเลี่ยง

❌ doBooking()

❌ updateStatus() (กำกวม)

2.3 Variable Naming

มาตรฐาน

- ใช้ **camelCase**
- ชื่ออธิบายความหมายได้ชัดเจน
- ห้ามย่อโดยไม่จำเป็น

ตัวอย่าง

userId

lockId

bookingId

paymentId

bookingDate

approvedByUserId

rejectedReason

paymentStatus

createdAt

updatedAt

หลีกเลี่ยง

❌ uid

❌ bid

❌ dt

3. Database Naming Convention

3.1 Table Naming

มาตรฐาน

- ใช้ **snake_case**
- ใช้คำนามเอกพจน์
- ชื่อสื่อความหมายชัดเจน

ตัวอย่าง

app_user

market_lock

booking

payment

3.2 Column Naming

มาตรฐาน

- snake_case
- ลงท้ายด้วย _id สำหรับ Primary/Foreign Key

ตัวอย่าง

user_id

lock_id

booking_date

approved_by

payment_status

created_at

3.3 Key Standards

- Primary Key ต้องเป็น {table_name}_id
- Foreign Key ต้องตรงกับชื่อ Primary Key ต้นทาง

ตัวอย่าง:

booking.user_id ⟶ app_user.user_id

booking.lock_id ⟶ market_lock.lock_id

payment.booking_id ⟶ booking.booking_id

4. Code Structure Standard

4.1 Layered Architecture

ระบบต้องแบ่งเป็น Layer ชัดเจน

Controller Layer

Service Layer

Repository Layer

Database Layer

ตัวอย่างโครงสร้างไฟล์

controllers/

services/

repositories/

models/

4.2 Separation of Responsibility

- Controller ⟶ รับ Request / ส่ง Response
- Service ⟶ Business Logic
- Repository ⟶ ติดต่อ Database
- Database ⟶ Constraint และ Index

ห้ามเขียน SQL ตรงใน Controller

5. Commenting Standard

หลักการสำคัญ

เขียน WHY ไม่ใช่ WHAT

ตัวอย่างที่ถูกต้อง

// ตรวจสอบ role เพราะเฉพาะ manager/admin เท่านั้นที่สามารถอนุมัติการจองได้

ตัวอย่างที่ไม่ควรทำ

// อัปเดตสถานะเป็น approved

(อ่านจากโค้ดก็รู้แล้ว)

6. Error Handling Standard

- ต้องจัดการ Error ทุกจุดที่มีการติดต่อ Database
- ห้ามปล่อย Raw Error ไปยัง User
- ต้องใช้ Error Message ที่เข้าใจง่าย

ตัวอย่าง:

"Lock is already booked for this date."

"Unauthorized access."

7. Business Rule Enforcement

7.1 Database-Level Enforcement

ระบบใช้ PostgreSQL Constraint เพื่อควบคุมกฎสำคัญ เช่น:

- CHECK constraint (role, status)
- Partial Unique Index ป้องกันจองซ้ำ

ตัวอย่าง:

```
CREATE UNIQUE INDEX uq_booking_active
ON booking(lock_id, booking_date)
WHERE status IN ('pending','approved');
```

หลักการ

ห้ามเขียน Logic ซ้ำใน Code ถ้า Database ควบคุมแล้ว

8. Security Standards

- Password ต้อง Hash ก่อนบันทึก
- ห้ามเก็บ Plain Text Password
- ใช้ Role-Based Access Control (RBAC)
- ตรวจสอบสิทธิ์ก่อน approve / reject

9. Date & Time Handling

- ใช้ TIMESTAMPTZ เสมอ
- ใช้ UTC ใน Backend
- แปลง Timezone เฉพาะฝั่ง Frontend

10. Performance Standards

- ใช้ Index ตามที่ออกแบบใน Schema
- หลีกเลี่ยง SELECT *
- ใช้ Pagination ใน listBookings()

11. Code Formatting Rules

- Indentation: 2 หรือ 4 spaces (ต้องเลือกแบบเดียวทั้งโปรเจกต์)
- ใช้ Prettier / ESLint (ถ้าใช้ Node.js)
- ห้าม commit โค้ดที่ format ไม่ตรงมาตรฐาน

12. Version Control Standard

- ใช้ Branch แยก Feature
- ห้าม commit ตรงเข้า main
- Commit message ต้องอธิบายชัดเจน

ตัวอย่าง:

feat: add booking approval logic

fix: prevent duplicate active booking

refactor: improve payment verification flow

13. Testing Standard (ถ้ามี)

- ต้องมี Test สำหรับ:
    - createBooking
    - approveBooking
    - rejectBooking
    - payment verification
- ทดสอบกรณีจองซ้ำ